OPERATION KOMOREBI

Komorebi Circuit Obstructions

Lean-Verified Bounds

Block 02: degree with sharing
and complete safe-cover restoration

Komorebi P.

Local source release v1.2.0 • September 21, 2026

115 checked declarations, including 75 added in this block

The cumulative formalization covers C01, the maintained NCT degree dependency, and the lower-bound component of C02. C02's evaluation-span dual and C03–C08 are not formalized by this block.

Definitions, proof arguments, exact declaration register, source locations, axiom dependencies, and verification boundaries. No resolution of P versus NP or independent mathematical review is claimed.

The verified result

Let c be a finite legal classical NOT/CNOT/Toffoli circuit. For n input bits and a zero-initialized ancillas, assume for every x and both initial target values b that

$$\mathrm{run}(c)(x, 0^a, b) = (x, 0^a, b \oplus f(x)).$$

Write $T = \mathrm{tc}(c)$ for its number of Toffoli gates. The new joint endpoint is

Degree and complete-cover bound

$$\boxed{\mathrm{DegreeLE}(T+1, f) \quad \text{and} \quad 2\beta^*(f) \le T+3.}$$

`CircuitObstructions.restored_degree_and_cover_bound`

The first conclusion is an explicit certificate that f has a Boolean polynomial representation of degree at most $T+1$. The second is the affine-optimized complete safe-cover lower bound, equivalently $T \ge \max\{0, 2\beta^*(f)-3\}$.

This applies even when the output controls intermediate gates, the inputs are modified temporarily, and workspace is dirty internally. It requires restoration only at the boundary. It makes no restriction to clean intermediate representations.

What changed

Block 01 proved that discrepancies between a prefix and an inverse suffix form a complete safe cover. Block 02 supplies the missing degree dependency, proves that the cover parameters exist, constructs the balanced cut, and implements affine-output adjustment using legal zero-Toffoli circuits.

What did not change

The mathematical scope remains the eight maintained claim groups. C02 is still partial as a group: its dual-certificate theorem is not included. The formal source also proves

$$\beta(f) \le \beta^*(f) \le n.$$

This ceiling prevents the degree parameter alone from producing a superpolynomial lower bound. No new best-known bound or publication-novelty claim is made.

Where the proof lives

Every proof term is in `formalization/lean/CircuitObstructions/`. The source inventory, fresh elaboration log, and axiom report identify the exact statements. The register in this document lists all 115 maintained theorem declarations, including the 40 inherited declarations. Imported standard-library theorems are not counted.

Model and definitions

THE ASSUMPTIONS ARE PART OF THE THEOREM

A register state is a Boolean function on a finite wire set. The canonical set has n input wires, a ancillas, and one target. A NOT complements a bit. A CNOT adds one control to a distinct target. A Toffoli adds the product of two distinct controls to a third distinct wire. Gate constructors carry the required inequalities.

All arithmetic is over $\mathbb{F}_2$, implemented using Boolean XOR and AND. Each gate is its own inverse. Circuit reversal, restoration, and every cut are inherited from the unchanged Block 01 source.

Quantity	Meaning in the formal source
T	Toffoli count. NOT and CNOT have cost zero in this count only.
DegreeLE(d, f)	f is an XOR of finitely many monomials, each containing at most d variable occurrences.
Safe detector	A Boolean h with $h(x) = 1 \Rightarrow f(x) = 1$.
Complete coverage	Every positive v has a safe detector with $h(v) = 1$.
$\beta(f)$	Least degree bound sufficient for complete safe coverage.
$\beta^*(f)$	Least upper bound on $\beta(f \oplus g)$ over all degree-one g.

Polynomial certificates are not postulated degree oracles

The inductive certificate has three constructors: zero, a bounded-length monomial, and XOR of two certified functions. It is tied to the concrete evaluator

$$\text{polyEval}(p, x) = \bigoplus_{L \in p} \prod_{i \in L} x_i$$

by a proved equivalence. A certificate yields an explicit finite monomial list, and such a list yields a certificate.

Repeated variables are allowed in the list. Boolean idempotence reduces their repetitions without increasing degree. This is the representation form of the usual Boolean degree bound. The formal source does not separately implement canonical ANF normalization or claim that extracting every homogeneous grade is free.

Boundary of the model

There are no quantum phases, measurements, or discarded work registers. Inputs and zero ancillas restore for every logical input and both target values. Standard classical choice defines least semantic bounds, but supplies no polynomial-time procedure for finding their witnesses.

Why sharing does not break the degree bound

A STRONGER INVARIANT THAN SINGLE-REGISTER DEGREE

For a current state $s(x)$ after t Toffolis, the invariant is

$$\forall L, \qquad \mathrm{DegreeLE}\left(t + |L|,\ x \mapsto \prod_{i \in L} s_i(x)\right).$$

L may be any finite list, including repeated registers. Initially each coordinate is a variable or a constant, so a product of r coordinates has degree at most r.

Suppose the next gate toggles target u by the product of a control list of length q. A product that omits u is unchanged. If u occurs, idempotence removes all its repetitions. With R the remaining list, the updated product is

$$\left(s_u \prod_{i \in R} s_i\right) \oplus \left(\prod_{j \in \mathrm{controls}} s_j \prod_{i \in R} s_i\right).$$

The two old-state products have certified degrees at most

$$t + 1 + |R| \qquad \text{and} \qquad t + q + |R|.$$

Since $|R| + 1 \le |L|$, the invariant's budget rises by at most $\max\{q - 1, 0\}$: zero for NOT and CNOT, one for Toffoli.

Induction along the actual gate list gives

$$\mathrm{DegreeLE}\left(T + r,\ x \mapsto \prod_{i \in L} \mathrm{run}(c)(s(x))_i\right), \qquad r = |L|.$$

With $r = 1$, every register has degree at most $T + 1$.

Why this handles a DAG rather than just a formula

The proof never unfolds a reused intermediate value into duplicated computation. It estimates arbitrary products of the current registers in one shared state. The budget increases once per actual Toffoli, including when its controls share earlier work or its target participates in feedback.

The statements are `workspaceDegree_run` and `canonical_run_coordinate_degree`. This formalizes the maintained NCT specialization of F3. A separate generic XOR–AND circuit syntax and simulation theorem are not added here.

From the cut to a numerical lower bound

EXISTENCE, BALANCE, AND COMPLETE COVERAGE

Write $c = p \mathbin{+\!\!+} q$. At this cut, Block 01 defines

$$H_i(x) = \mathrm{run}(p)(x, 0^a, 0)_i \oplus \mathrm{run}(q^{-1})(x, 0^a, 0)_i$$

and proves

$$H_i \le f, \qquad f(x) = 1 \iff \exists i\ H_i(x) = 1.$$

Reversal does not change Toffoli count. The new coordinate degree theorem therefore gives

$$\deg(H_i) \le \max\{\mathrm{tc}(p), \mathrm{tc}(q)\} + 1.$$

The least safe degree exists

For a positive v, take the point indicator

$$\delta_v(x) = \prod_{i=1}^{n} \begin{cases} x_i, & v_i = 1, \\ 1 + x_i, & v_i = 0. \end{cases}$$

It is safe, detects v, and has a degree-n certificate. Hence complete safe coverage exists at degree n for every function. Strong induction proves the existence of a least admissible natural degree. The definition `beta` chooses this bound and its specification and minimality are proved.

The balanced cut is constructed

Because each gate has cost zero or one, a prefix can realize every Toffoli count from zero to T. The source proves this fact and extracts a cut with

$$\max\{\mathrm{tc}(p), \mathrm{tc}(q)\} \le \lceil T/2 \rceil .$$

The complete safe cut cover then proves

$$\beta(f) \le \lceil T/2 \rceil + 1, \qquad \boxed{2\beta(f) \le T + 3.}$$

Natural-number arithmetic retains the floor/ceiling and small-T cases. No balance assumption is added to the final theorem.

Affine output adjustment is an actual circuit

THE OPTIMIZATION DOES NOT ASSUME A CLEAN NORMAL FORM

For every degree-one Boolean function g, the source constructs a restored implementation with zero Toffolis.

A constant is computed by the empty circuit or target NOT. An input variable uses a legal input-to-target CNOT. XOR is implemented by concatenating restored circuits. Induction on a degree-one certificate gives the general construction.

If c restores f and d restores g, their concatenation restores $f \oplus g$. For the affine implementation,

$$\mathrm{tc}(c \mathbin{+\!\!+} d) = \mathrm{tc}(c).$$

The unoptimized bound applies to every affine g at the same T. Define $\beta^*(f)$ as the least upper bound of those $\beta(f \oplus g)$ values. Existence follows from their common upper bound n.

Therefore

$$\boxed{2\beta^*(f) \le T + 3, \qquad \beta(f) \le \beta^*(f) \le n.}$$

The final output at initial target zero is exactly f, so the forward coordinate theorem also gives $\mathrm{DegreeLE}(T+1, f)$. The joint endpoint combines these two proved conclusions.

Nonvacuity: two-input AND

Every degree-one function on two inputs has zero XOR over the four corners of the input square. AND has corner XOR one. Thus AND has no degree-one certificate.

A degree-one complete safe cover for AND would include a detector of its sole accepting input. Safety forces that detector to be AND itself, contradicting the corner-parity lemma. The degree-two monomial is a valid detector. The source proves

$$\beta(\mathrm{AND}_2) = \beta^*(\mathrm{AND}_2) = 2$$

and therefore every restored implementation of AND_2 uses at least one Toffoli.

This is a theorem about all implementations in the model, not an exhaustive search through circuits. It does not formalize the general k-input monomial optimum.

Verification and proof boundaries

UNIVERSAL PROOFS AND FINITE CONTROLS REMAIN DISTINCT

Module	Count	Formal role
Gates	16	Inherited legal primitives and reversal
CutComparison	13	Inherited complete cut cover
Examples	11	Inherited dirty and output-active controls
Degree	19	Polynomial certificates and closure
MultiplicativeDegree	12	Degree invariant with sharing
SafeDegree	22	Least complete degree and balanced cuts
AffineOutput	14	Affine compiler and optimized bound
DegreeChecks	8	AND2 exclusion and exact parameters
Total	115	75 new declarations, 40 inherited

The recorded audit builds all source in a fresh project directory, treats warnings as errors, prints each declaration's type and axiom dependencies, and replays all eight modules through the bundled checker. Eight deliberately invalid proof controls are required to be rejected.

Only `propext`, `Classical.choice`, and `Quot.sound` occur in the audited dependencies. No custom axiom, admitted proof, native-decision oracle, or kernel bypass is used. Lean 4.34.0-rc2 and its bundled Std are pinned. This checker shares the toolchain's trust boundary and is not an independent implementation or human review.

The separate Python checker uses exact truth tables and integer Möbius transforms. It covers all 278 functions on zero through three inputs; 2,698 legal circuit words in specified width/length ranges; 28,088 register-product checks; and 251 restored words. The universal theorems do not follow by extrapolation from those finite tests.

What remains outside the formal block

C02's evaluation-span dual, affine input invariance, and general monotone ideal are not formalized. C03–C08 retain their previous written and finite-check status. The original classical/reversible circuit-size comparison and general exact monomial-cost theorem are not newly formalized. The active claim register controls interpretation.

Reproduction, provenance, and the declaration register

A REPLAYABLE LOCAL RELEASE

From the repository root, with the pinned toolchain installed:

```
python scripts/verify_lean.py --toolchain /path/to/toolchain
python scripts/verify.py --full
```

The first command performs the fresh formal audit. The second reproduces the seven original frozen scientific outputs and the new independent degree controls. No toolchain or package downloads occur in these commands. Only Linux x86_64 execution is recorded.

Provenance and development failures

The parent release is v1.1.0, commit `ab5010d0eba21f3711e00ed1eaa3268dee9ee56b`. The original gate, cut, and example modules remain byte-identical. Their 40 declarations are rechecked along with the new files; predecessor logs are not substituted for the new build.

A download probe failed, and a rejected streaming launch did not start a compiler. The core toolchain was recovered from a CRC-checked first portable archive part. Its executable hashes match the predecessor. The partial XZ/TAR stream ended at the multipart boundary, so no complete upstream-distribution integrity claim is made.

Early Lean attempts used unavailable library names or incomplete unfolding and encountered strict-warning failures. These were corrected without admitted proofs. A draft README link to the not-yet-created PDF was also caught. The retained development log distinguishes those failures from the expected rejection controls and from the final passing audit.

How to read the register

All 115 entries below come from the fresh elaboration log and the checked source inventory. Each entry identifies its source file, line, and axiom dependencies. The 40 inherited entries remain included so the reference is cumulative. Definitions are explained above and remain in the source; imported library declarations are not listed as project results.

For monospace readability, logical glyphs in printed signatures are rendered in ASCII: `forall`, `exists`, `->`, `<->`, `<=`, `!=`, and `/\`. Lean's `^^` denotes Boolean XOR. These are editorial glyph substitutions, not a claim that the printed text is standalone compilable source. Unmodified UTF-8 signatures are retained in `signatures_utf8.txt` and `declaration_catalog.json`.

The Lean files and elaborated types are authoritative if any explanatory rendering differs. The original literature references remain in `docs/REFERENCES.md`; this supplement makes no novelty claim.

Declaration register: Gates

16 DECLARATIONS

Inherited legal gate semantics, reversal, and gate-count identities. These source bytes are unchanged from v1.1.0.

L001 CircuitObstructions.write_same

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 17.

```
CircuitObstructions.write_same {W : Type} [DecidableEq W] (s : CircuitObstructions.State W) (t : W) (b : Bool) :
  CircuitObstructions.write s t b t = b
```

Axiom dependencies: propext.

L002 CircuitObstructions.write_other

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 20.

```
CircuitObstructions.write_other {W : Type} [DecidableEq W] (s : CircuitObstructions.State W) (t i : W) (b : Bool)
  (h : i != t) : CircuitObstructions.write s t b i = s i
```

Axiom dependencies: propext.

L003 CircuitObstructions.xor_cancel_right

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 23.

```
CircuitObstructions.xor_cancel_right (a b : Bool) : (a ^^ b ^^ b) = a
```

Axiom dependencies: none.

L004 CircuitObstructions.xor_cancel_left

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 26.

```
CircuitObstructions.xor_cancel_left (a b : Bool) : (a ^^ (a ^^ b)) = b
```

Axiom dependencies: none.

L005 CircuitObstructions.xor_commute

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 29.

```
CircuitObstructions.xor_commute (a b : Bool) : (a ^^ b) = (b ^^ a)
```

Axiom dependencies: none.

L006 CircuitObstructions.xor_true_iff_ne

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 32.

```
CircuitObstructions.xor_true_iff_ne (a b : Bool) : (a ^^ b) = true <-> a != b
```

Axiom dependencies: none.

Gates (continued)

L007 CircuitObstructions.Gate.eval_involutive

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 48.

```
CircuitObstructions.Gate.eval_involutive {W : Type} [DecidableEq W] (g : CircuitObstructions.Gate W)
  (s : CircuitObstructions.State W) : g.eval (g.eval s) = s
```

Axiom dependencies: propext, Quot.sound.

L008 CircuitObstructions.Gate.eval_injective

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 67.

```
CircuitObstructions.Gate.eval_injective {W : Type} [DecidableEq W] (g : CircuitObstructions.Gate W)
  {s t : CircuitObstructions.State W} (h : g.eval s = g.eval t) : s = t
```

Axiom dependencies: propext, Quot.sound.

L009 CircuitObstructions.run_nil

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 78.

```
CircuitObstructions.run_nil {W : Type} [DecidableEq W] (s : CircuitObstructions.State W) :
  CircuitObstructions.run [] s = s
```

Axiom dependencies: none.

L010 CircuitObstructions.run_cons

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 81.

```
CircuitObstructions.run_cons {W : Type} [DecidableEq W] (g : CircuitObstructions.Gate W)
  (gs : CircuitObstructions.Circuit W) (s : CircuitObstructions.State W) :
  CircuitObstructions.run (g :: gs) s = CircuitObstructions.run gs (g.eval s)
```

Axiom dependencies: none.

L011 CircuitObstructions.run_append

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 84.

```
CircuitObstructions.run_append {W : Type} [DecidableEq W] (p q : CircuitObstructions.Circuit W)
  (s : CircuitObstructions.State W) :
  CircuitObstructions.run (p ++ q) s = CircuitObstructions.run q (CircuitObstructions.run p s)
```

Axiom dependencies: none.

Gates (continued)

L012 CircuitObstructions.run_reverse_run

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 90.

```
CircuitObstructions.run_reverse_run {W : Type} [DecidableEq W] (c : CircuitObstructions.Circuit W)
  (s : CircuitObstructions.State W) : CircuitObstructions.run (List.reverse c) (CircuitObstructions.run c s) = s
```

Axiom dependencies: propext, Quot.sound.

L013 CircuitObstructions.run_run_reverse

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 98.

```
CircuitObstructions.run_run_reverse {W : Type} [DecidableEq W] (c : CircuitObstructions.Circuit W)
  (s : CircuitObstructions.State W) : CircuitObstructions.run c (CircuitObstructions.run (List.reverse c) s) = s
```

Axiom dependencies: propext, Quot.sound.

L014 CircuitObstructions.run_injective

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 102.

```
CircuitObstructions.run_injective {W : Type} [DecidableEq W] (c : CircuitObstructions.Circuit W)
  {s t : CircuitObstructions.State W} (h : CircuitObstructions.run c s = CircuitObstructions.run c t) : s = t
```

Axiom dependencies: propext, Quot.sound.

L015 CircuitObstructions.toffoliCount_append

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 114.

```
CircuitObstructions.toffoliCount_append {W : Type} (p q : CircuitObstructions.Circuit W) :
  CircuitObstructions.toffoliCount (p ++ q) = CircuitObstructions.toffoliCount p + CircuitObstructions.toffoliCount q
```

Axiom dependencies: propext.

L016 CircuitObstructions.toffoliCount_reverse

Inherited from Block 01. Source: CircuitObstructions/Gates.lean, line 118.

```
CircuitObstructions.toffoliCount_reverse {W : Type} (c : CircuitObstructions.Circuit W) :
  CircuitObstructions.toffoliCount (List.reverse c) = CircuitObstructions.toffoliCount c
```

Axiom dependencies: propext.

Declaration register: CutComparison

13 DECLARATIONS

Inherited boundary and cut-comparison results. The generic statement is specialized to the finite canonical input, zero-ancilla, and target registers.

L017 CircuitObstructions.Boundary.target_separated

Inherited from Block 01. Source: `CircuitObstructions/CutComparison.lean`, line 17.

```
CircuitObstructions.Boundary.target_separated {X W : Type} (e : CircuitObstructions.Boundary X W) (x : X) :
  e.encode x false != e.encode x true
```

Axiom dependencies: `propext`.

L018 CircuitObstructions.canonical_input

Inherited from Block 01. Source: `CircuitObstructions/CutComparison.lean`, line 39.

```
CircuitObstructions.canonical_input (n a : Nat) (x : Fin n -> Bool) (b : Bool) (i : Fin n) :
  (CircuitObstructions.canonicalBoundary n a).encode x b (Sum.inl i) = x i
```

Axiom dependencies: none.

L019 CircuitObstructions.canonical_ancilla

Inherited from Block 01. Source: `CircuitObstructions/CutComparison.lean`, line 42.

```
CircuitObstructions.canonical_ancilla (n a : Nat) (x : Fin n -> Bool) (b : Bool) (i : Fin a) :
  (CircuitObstructions.canonicalBoundary n a).encode x b (Sum.inr (Sum.inl i)) = false
```

Axiom dependencies: none.

L020 CircuitObstructions.canonical_target

Inherited from Block 01. Source: `CircuitObstructions/CutComparison.lean`, line 45.

```
CircuitObstructions.canonical_target (n a : Nat) (x : Fin n -> Bool) (b : Bool) :
  (CircuitObstructions.canonicalBoundary n a).encode x b (Sum.inr (Sum.inr ())) = b
```

Axiom dependencies: none.

L021 CircuitObstructions.inverse_suffix_boundary

Inherited from Block 01. Source: `CircuitObstructions/CutComparison.lean`, line 49.

```
CircuitObstructions.inverse_suffix_boundary {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X -> Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) (x : X)
  (b : Bool) :
  CircuitObstructions.run (List.reverse q) (e.encode x b) = CircuitObstructions.run p (e.encode x (b ^^ f x))
```

Axiom dependencies: `propext`, `Quot.sound`.

CutComparison (continued)

L022 CircuitObstructions.direction_nonzero

Inherited from Block 01. Source: CircuitObstructions/CutComparison.lean, line 67.

```
CircuitObstructions.direction_nonzero {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p : CircuitObstructions.Circuit W) (x : X) : exists i, CircuitObstructions.direction e p x i = true
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L023 CircuitObstructions.cut_transport_identity

Inherited from Block 01. Source: CircuitObstructions/CutComparison.lean, line 82.

```
CircuitObstructions.cut_transport_identity {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X -> Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) (x : X)
  (i : W) : CircuitObstructions.packet e p q x i = (f x && CircuitObstructions.direction e p x i)
```

Axiom dependencies: propext, Quot.sound.

L024 CircuitObstructions.cut_packet_safe

Inherited from Block 01. Source: CircuitObstructions/CutComparison.lean, line 91.

```
CircuitObstructions.cut_packet_safe {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X -> Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) (x : X)
  (i : W) : CircuitObstructions.packet e p q x i = true -> f x = true
```

Axiom dependencies: propext, Quot.sound.

L025 CircuitObstructions.cut_cover_complete

Inherited from Block 01. Source: CircuitObstructions/CutComparison.lean, line 99.

```
CircuitObstructions.cut_cover_complete {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X -> Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) (x : X) :
  f x = true <-> exists i, CircuitObstructions.packet e p q x i = true
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

CutComparison (continued)

L026 CircuitObstructions.complete_safe_cover

Inherited from Block 01. Source: `CircuitObstructions/CutComparison.lean`, line 111.

```
CircuitObstructions.complete_safe_cover {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X -> Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) :
  (forall (x : X) (i : W), CircuitObstructions.packet e p q x i = true -> f x = true) /\
    forall (x : X), f x = true <-> exists i, CircuitObstructions.packet e p q x i = true
```

Axiom dependencies: `propext`, `Classical.choice`, `Quot.sound`.

L027 CircuitObstructions.cut_states_equal_iff_reject

Inherited from Block 01. Source: `CircuitObstructions/CutComparison.lean`, line 119.

```
CircuitObstructions.cut_states_equal_iff_reject {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (p q : CircuitObstructions.Circuit W) (f : X -> Bool) (hR : CircuitObstructions.Restores e (p ++ q) f) (x : X) :
  CircuitObstructions.run p (e.encode x false) = CircuitObstructions.run (List.reverse q) (e.encode x false) <->
    f x = false
```

Axiom dependencies: `propext`, `Quot.sound`.

L028 CircuitObstructions.every_index_cut

Inherited from Block 01. Source: `CircuitObstructions/CutComparison.lean`, line 131.

```
CircuitObstructions.every_index_cut {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (c : CircuitObstructions.Circuit W) (f : X -> Bool) (hR : CircuitObstructions.Restores e c f) (k : Nat) (x : X) :
  f x = true <-> exists i, CircuitObstructions.packet e (List.take k c) (List.drop k c) x i = true
```

Axiom dependencies: `propext`, `Classical.choice`, `Quot.sound`.

L029 CircuitObstructions.canonical_complete_safe_cover

Inherited from Block 01. Source: `CircuitObstructions/CutComparison.lean`, line 139.

```
CircuitObstructions.canonical_complete_safe_cover (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : (Fin n -> Bool) -> Bool)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) (k : Nat) :
  (forall (x : Fin n -> Bool) (i : CircuitObstructions.CanonicalWire n a),
      CircuitObstructions.packet (CircuitObstructions.canonicalBoundary n a) (List.take k c) (List.drop k c) x i =
          true ->
        f x = true) /\
    forall (x : Fin n -> Bool),
      f x = true <->
        exists i,
          CircuitObstructions.packet (CircuitObstructions.canonicalBoundary n a) (List.take k c) (List.drop k c) x i
          ↪  =
            true
```

Axiom dependencies: `propext`, `Classical.choice`, `Quot.sound`.

Declaration register: Examples

11 DECLARATIONS

Inherited dirty-workspace and output-active controls. These are actual legal circuits, not assumptions about a computation.

L030 CircuitObstructions.Examples.andOracle_restores

Inherited from Block 01. Source: CircuitObstructions/Examples.lean, line 27.

```
CircuitObstructions.Examples.andOracle_restores :
  CircuitObstructions.Restores CircuitObstructions.Examples.andBoundary CircuitObstructions.Examples.andOracle fun x
  ↪  =>
    x.fst && x.snd
```

Axiom dependencies: Quot.sound.

L031 CircuitObstructions.Examples.andOracle_toffoli_count

Inherited from Block 01. Source: CircuitObstructions/Examples.lean, line 33.

```
CircuitObstructions.Examples.andOracle_toffoli_count :
  CircuitObstructions.toffoliCount CircuitObstructions.Examples.andOracle = 2
```

Axiom dependencies: propext.

L032 CircuitObstructions.Examples.and_scratch_can_be_dirty

Inherited from Block 01. Source: CircuitObstructions/Examples.lean, line 36.

```
CircuitObstructions.Examples.and_scratch_can_be_dirty :
  CircuitObstructions.run CircuitObstructions.Examples.andCompute
      (CircuitObstructions.Examples.andBoundary.encode (true, true) false)
      CircuitObstructions.Examples.WorkWire.scratch =
    true
```

Axiom dependencies: none.

L033 CircuitObstructions.Examples.andOracle_every_cut

Inherited from Block 01. Source: CircuitObstructions/Examples.lean, line 40.

```
CircuitObstructions.Examples.andOracle_every_cut (k : Nat) (x : Bool x Bool) :
  (x.fst && x.snd) = true <->
    exists i,
      CircuitObstructions.packet CircuitObstructions.Examples.andBoundary
          (List.take k CircuitObstructions.Examples.andOracle) (List.drop k CircuitObstructions.Examples.andOracle) x
          i =
        true
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L034 CircuitObstructions.Examples.outputActiveOracle_restores

Inherited from Block 01. Source: CircuitObstructions/Examples.lean, line 66.

```
CircuitObstructions.Examples.outputActiveOracle_restores :
  CircuitObstructions.Restores CircuitObstructions.Examples.swapBoundary
  ↪  CircuitObstructions.Examples.outputActiveOracle
    fun x => true
```

Axiom dependencies: Quot.sound.

L035 CircuitObstructions.Examples.outputActiveOracle_length

Inherited from Block 01. Source: CircuitObstructions/Examples.lean, line 71.

```
CircuitObstructions.Examples.outputActiveOracle_length : List.length CircuitObstructions.Examples.outputActiveOracle
↪  = 7
```

Axiom dependencies: none.

L036 CircuitObstructions.Examples.outputActiveOracle_toffoli_count

Inherited from Block 01. Source: CircuitObstructions/Examples.lean, line 73.

```
CircuitObstructions.Examples.outputActiveOracle_toffoli_count :
  CircuitObstructions.toffoliCount CircuitObstructions.Examples.outputActiveOracle = 0
```

Axiom dependencies: propext.

L037 CircuitObstructions.Examples.moved_discrepancy_data

Inherited from Block 01. Source: CircuitObstructions/Examples.lean, line 76.

```
CircuitObstructions.Examples.moved_discrepancy_data (x : Bool) :
  CircuitObstructions.packet CircuitObstructions.Examples.swapBoundary CircuitObstructions.Examples.swapCircuit
      CircuitObstructions.Examples.swapSuffix x CircuitObstructions.Examples.SwapWire.data =
    true
```

Axiom dependencies: none.

L038 CircuitObstructions.Examples.moved_discrepancy_target

Inherited from Block 01. Source: CircuitObstructions/Examples.lean, line 80.

```
CircuitObstructions.Examples.moved_discrepancy_target (x : Bool) :
  CircuitObstructions.packet CircuitObstructions.Examples.swapBoundary CircuitObstructions.Examples.swapCircuit
      CircuitObstructions.Examples.swapSuffix x CircuitObstructions.Examples.SwapWire.target =
    false
```

Axiom dependencies: none.

L039 CircuitObstructions.Examples.target_only_reconstruction_is_false

Inherited from Block 01. Source: CircuitObstructions/Examples.lean, line 85.

```
CircuitObstructions.Examples.target_only_reconstruction_is_false :
  not forall (x : Bool),
      CircuitObstructions.packet CircuitObstructions.Examples.swapBoundary CircuitObstructions.Examples.swapCircuit
          CircuitObstructions.Examples.swapSuffix x CircuitObstructions.Examples.SwapWire.target =
        true
```

Axiom dependencies: none.

L040 CircuitObstructions.Examples.degenerate_encoding_control

Inherited from Block 01. Source: CircuitObstructions/Examples.lean, line 93.

```
CircuitObstructions.Examples.degenerate_encoding_control :
  have bad := fun x x_1 => false;
  (forall (y : Bool), CircuitObstructions.run [] (bad y) = bad (y ^^ true)) /\
    forall (i : CircuitObstructions.Examples.SwapWire), (bad false i ^^ bad false i) = false
```

Axiom dependencies: none.

Declaration register: Degree

19 DECLARATIONS

New polynomial-representation certificates over Boolean functions. Repeated monomial variables are permitted, and the certificate is explicitly connected to XOR-list polynomial evaluation.

L041 CircuitObstructions.bprod_nil

Block 02. Source: CircuitObstructions/Degree.lean, line 19.

```
CircuitObstructions.bprod_nil {W : Type} (s : CircuitObstructions.State W) : CircuitObstructions.bprod [] s = true
```

Axiom dependencies: none.

L042 CircuitObstructions.bprod_cons

Block 02. Source: CircuitObstructions/Degree.lean, line 20.

```
CircuitObstructions.bprod_cons {W : Type} (i : W) (is : List W) (s : CircuitObstructions.State W) :
  CircuitObstructions.bprod (i :: is) s = (s i && CircuitObstructions.bprod is s)
```

Axiom dependencies: none.

L043 CircuitObstructions.bprod_append

Block 02. Source: CircuitObstructions/Degree.lean, line 23.

```
CircuitObstructions.bprod_append {W : Type} (l k : List W) (s : CircuitObstructions.State W) :
  CircuitObstructions.bprod (l ++ k) s = (CircuitObstructions.bprod l s && CircuitObstructions.bprod k s)
```

Axiom dependencies: propext.

L044 CircuitObstructions.xor_and_right

Block 02. Source: CircuitObstructions/Degree.lean, line 29.

```
CircuitObstructions.xor_and_right (a b c : Bool) : ((a ^^ b) && c) = (a && c ^^ b && c)
```

Axiom dependencies: none.

L045 CircuitObstructions.and_xor_right

Block 02. Source: CircuitObstructions/Degree.lean, line 33.

```
CircuitObstructions.and_xor_right (a b c : Bool) : (a && (b ^^ c)) = (a && b ^^ a && c)
```

Axiom dependencies: none.

L046 CircuitObstructions.not_as_xor_one

Block 02. Source: CircuitObstructions/Degree.lean, line 37.

```
CircuitObstructions.not_as_xor_one (a : Bool) : (!a) = (a ^^ true)
```

Axiom dependencies: none.

L047 CircuitObstructions.DegreeLE.mono

Block 02. Source: CircuitObstructions/Degree.lean, line 47.

```
CircuitObstructions.DegreeLE.mono {n d e : Nat} {f : CircuitObstructions.BFun n} (h : CircuitObstructions.DegreeLE d
↪  f)
  (hde : d <= e) : CircuitObstructions.DegreeLE e f
```

Axiom dependencies: none.

L048 CircuitObstructions.DegreeLE.one

Block 02. Source: CircuitObstructions/Degree.lean, line 54.

```
CircuitObstructions.DegreeLE.one (n d : Nat) : CircuitObstructions.DegreeLE d fun x => true
```

Axiom dependencies: none.

L049 CircuitObstructions.DegreeLE.constant

Block 02. Source: CircuitObstructions/Degree.lean, line 57.

```
CircuitObstructions.DegreeLE.constant (n d : Nat) (b : Bool) : CircuitObstructions.DegreeLE d fun x => b
```

Axiom dependencies: none.

L050 CircuitObstructions.DegreeLE.variable

Block 02. Source: CircuitObstructions/Degree.lean, line 63.

```
CircuitObstructions.DegreeLE.variable {n : Nat} (i : Fin n) : CircuitObstructions.DegreeLE 1 fun x => x i
```

Axiom dependencies: propext, Quot.sound.

L051 CircuitObstructions.DegreeLE.xor

Block 02. Source: CircuitObstructions/Degree.lean, line 67.

```
CircuitObstructions.DegreeLE.xor {n d e : Nat} {f g : CircuitObstructions.BFun n}
  (hf : CircuitObstructions.DegreeLE d f) (hg : CircuitObstructions.DegreeLE e g) :
  CircuitObstructions.DegreeLE (max d e) fun x => f x ^^ g x
```

Axiom dependencies: propext.

L052 CircuitObstructions.DegreeLE.not

Block 02. Source: CircuitObstructions/Degree.lean, line 72.

```
CircuitObstructions.DegreeLE.not {n d : Nat} {f : CircuitObstructions.BFun n} (hf : CircuitObstructions.DegreeLE d f)
↪  :
  CircuitObstructions.DegreeLE d fun x => !f x
```

Axiom dependencies: Quot.sound.

L053 CircuitObstructions.DegreeLE.monomial_mul

Block 02. Source: CircuitObstructions/Degree.lean, line 76.

```
CircuitObstructions.DegreeLE.monomial_mul {n d e : Nat} (l : List (Fin n)) (hl : l.length <= d)
  {g : CircuitObstructions.BFun n} (hg : CircuitObstructions.DegreeLE e g) :
  CircuitObstructions.DegreeLE (d + e) fun x => CircuitObstructions.bprod l x && g x
```

Axiom dependencies: propext, Quot.sound.

L054 CircuitObstructions.DegreeLE.mul

Block 02. Source: CircuitObstructions/Degree.lean, line 87.

```
CircuitObstructions.DegreeLE.mul {n d e : Nat} {f g : CircuitObstructions.BFun n}
  (hf : CircuitObstructions.DegreeLE d f) (hg : CircuitObstructions.DegreeLE e g) :
  CircuitObstructions.DegreeLE (d + e) fun x => f x && g x
```

Axiom dependencies: propext, Quot.sound.

L055 CircuitObstructions.DegreeLE.product

Block 02. Source: CircuitObstructions/Degree.lean, line 96.

```
CircuitObstructions.DegreeLE.product {n : Nat} {W : Type} (l : List W)
  (s : (Fin n -> Bool) -> CircuitObstructions.State W) (h : forall (i : W), CircuitObstructions.DegreeLE 1 fun x => s
  ↪  x i) :
  CircuitObstructions.DegreeLE l.length fun x => CircuitObstructions.bprod l (s x)
```

Axiom dependencies: propext, Quot.sound.

L056 CircuitObstructions.polyEval_append

Block 02. Source: CircuitObstructions/Degree.lean, line 109.

```
CircuitObstructions.polyEval_append {n : Nat} (p q : List (List (Fin n))) (x : Fin n -> Bool) :
  CircuitObstructions.polyEval (p ++ q) x = (CircuitObstructions.polyEval p x ^^ CircuitObstructions.polyEval q x)
```

Axiom dependencies: propext.

L057 CircuitObstructions.DegreeLE.polynomial_witness

Block 02. Source: CircuitObstructions/Degree.lean, line 115.

```
CircuitObstructions.DegreeLE.polynomial_witness {n d : Nat} {f : CircuitObstructions.BFun n}
  (h : CircuitObstructions.DegreeLE d f) :
  exists p, (forall (l : List (Fin n)), l in p -> l.length <= d) /\ CircuitObstructions.polyEval p = f
```

Axiom dependencies: propext, Quot.sound.

L058 CircuitObstructions.polynomial_degree_certificate

Block 02. Source: CircuitObstructions/Degree.lean, line 134.

```
CircuitObstructions.polynomial_degree_certificate {n d : Nat} (p : List (List (Fin n)))
  (hp : forall (l : List (Fin n)), l in p -> l.length <= d) : CircuitObstructions.DegreeLE d
  ↪  (CircuitObstructions.polyEval p)
```

Axiom dependencies: propext.

L059 CircuitObstructions.degreeLE_iff_polynomial

Block 02. Source: CircuitObstructions/Degree.lean, line 142.

```
CircuitObstructions.degreeLE_iff_polynomial {n d : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.DegreeLE d f <->
    exists p, (forall (l : List (Fin n)), l in p -> l.length <= d) /\ CircuitObstructions.polyEval p = f
```

Axiom dependencies: propext, Quot.sound.

Declaration register: MultiplicativeDegree

12 DECLARATIONS

New product invariant for register lists. This layer establishes the degree dependency while retaining arbitrary reuse and output feedback.

L060 CircuitObstructions.withoutWire_length_le

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 17.

```
CircuitObstructions.withoutWire_length_le {W : Type} [DecidableEq W] (l : List W) (t : W) :
  (CircuitObstructions.withoutWire l t).length <= l.length
```

Axiom dependencies: propext.

L061 CircuitObstructions.withoutWire_length_lt

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 21.

```
CircuitObstructions.withoutWire_length_lt {W : Type} [DecidableEq W] (l : List W) (t : W) (ht : t in l) :
  (CircuitObstructions.withoutWire l t).length + 1 <= l.length
```

Axiom dependencies: propext, Quot.sound.

L062 CircuitObstructions.bprod_write

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 33.

```
CircuitObstructions.bprod_write {W : Type} [DecidableEq W] (l : List W) (s : CircuitObstructions.State W) (t : W)
  (b : Bool) :
  CircuitObstructions.bprod l (CircuitObstructions.write s t b) =
    ((if t in l then b else true) && CircuitObstructions.bprod (CircuitObstructions.withoutWire l t) s)
```

Axiom dependencies: propext, Quot.sound.

L063 CircuitObstructions.bprod_write_absent

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 48.

```
CircuitObstructions.bprod_write_absent {W : Type} [DecidableEq W] (l : List W) (s : CircuitObstructions.State W) (t :
↪  W)
  (b : Bool) (ht : not t in l) :
  CircuitObstructions.bprod l (CircuitObstructions.write s t b) = CircuitObstructions.bprod l s
```

Axiom dependencies: propext.

L064 CircuitObstructions.workspaceDegree_initial

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 62.

```
CircuitObstructions.workspaceDegree_initial {n : Nat} {W : Type} (s : (Fin n -> Bool) -> CircuitObstructions.State W)
  (h : forall (i : W), CircuitObstructions.DegreeLE 1 fun x => s x i) : CircuitObstructions.WorkspaceDegree s 0
```

Axiom dependencies: propext, Quot.sound.

L065 CircuitObstructions.workspaceDegree_toggle

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 73.

```
CircuitObstructions.workspaceDegree_toggle {n : Nat} {W : Type} [DecidableEq W]
  (s : (Fin n -> Bool) -> CircuitObstructions.State W) (t : W) (controls : List W) (k : Nat)
  (h : CircuitObstructions.WorkspaceDegree s k) :
  CircuitObstructions.WorkspaceDegree (CircuitObstructions.toggleProduct s t controls) (k + (controls.length - 1))
```

Axiom dependencies: propext, Quot.sound.

L066 CircuitObstructions.workspaceDegree_gate

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 104.

```
CircuitObstructions.workspaceDegree_gate {n : Nat} {W : Type} [DecidableEq W]
  (s : (Fin n -> Bool) -> CircuitObstructions.State W) (g : CircuitObstructions.Gate W) (k : Nat)
  (h : CircuitObstructions.WorkspaceDegree s k) :
  CircuitObstructions.WorkspaceDegree (fun x => g.eval (s x)) (k + g.toffoliCost)
```

Axiom dependencies: propext, Quot.sound.

L067 CircuitObstructions.workspaceDegree_run

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 128.

```
CircuitObstructions.workspaceDegree_run {n : Nat} {W : Type} [DecidableEq W] (c : CircuitObstructions.Circuit W)
  (s : (Fin n -> Bool) -> CircuitObstructions.State W) (k : Nat) (h : CircuitObstructions.WorkspaceDegree s k) :
  CircuitObstructions.WorkspaceDegree (fun x => CircuitObstructions.run c (s x))
    (k + CircuitObstructions.toffoliCount c)
```

Axiom dependencies: propext, Quot.sound.

L068 CircuitObstructions.run_coordinate_degree

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 139.

```
CircuitObstructions.run_coordinate_degree {n : Nat} {W : Type} [DecidableEq W] (c : CircuitObstructions.Circuit W)
  (s : (Fin n -> Bool) -> CircuitObstructions.State W) (hs : forall (i : W), CircuitObstructions.DegreeLE 1 fun x =>
  ↪  s x i)
  (i : W) :
  CircuitObstructions.DegreeLE (CircuitObstructions.toffoliCount c + 1) fun x => CircuitObstructions.run c (s x) i
```

Axiom dependencies: propext, Quot.sound.

L069 CircuitObstructions.canonical_boundary_degree

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 146.

```
CircuitObstructions.canonical_boundary_degree (n a : Nat) (b : Bool) (i : CircuitObstructions.CanonicalWire n a) :
  CircuitObstructions.DegreeLE 1 fun x => (CircuitObstructions.canonicalBoundary n a).encode x b i
```

Axiom dependencies: propext, Quot.sound.

L070 CircuitObstructions.canonical_run_coordinate_degree

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 155.

```
CircuitObstructions.canonical_run_coordinate_degree (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (b : Bool)
  (i : CircuitObstructions.CanonicalWire n a) :
  CircuitObstructions.DegreeLE (CircuitObstructions.toffoliCount c + 1) fun x =>
    CircuitObstructions.run c ((CircuitObstructions.canonicalBoundary n a).encode x b) i
```

Axiom dependencies: propext, Quot.sound.

L071 CircuitObstructions.canonical_cut_packet_degree

Block 02. Source: CircuitObstructions/MultiplicativeDegree.lean, line 161.

```
CircuitObstructions.canonical_cut_packet_degree (n a : Nat)
  (p q : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a))
  (i : CircuitObstructions.CanonicalWire n a) :
  CircuitObstructions.DegreeLE (max (CircuitObstructions.toffoliCount p) (CircuitObstructions.toffoliCount q) + 1)
    fun x => CircuitObstructions.packet (CircuitObstructions.canonicalBoundary n a) p q x i
```

Axiom dependencies: propext, Quot.sound.

Declaration register: SafeDegree

22 DECLARATIONS

New complete safe-cover specification, least-degree existence, balanced Toffoli cuts, and the unoptimized restoration bound.

L072 CircuitObstructions.CompleteSafeDegreeLE.mono

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 16.

```
CircuitObstructions.CompleteSafeDegreeLE.mono {n d e : Nat} {f : CircuitObstructions.BFun n}
  (h : CircuitObstructions.CompleteSafeDegreeLE f d) (hde : d <= e) : CircuitObstructions.CompleteSafeDegreeLE f e
```

Axiom dependencies: none.

L073 CircuitObstructions.bprod_true_iff

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 22.

```
CircuitObstructions.bprod_true_iff {W : Type} (l : List W) (s : CircuitObstructions.State W) :
  CircuitObstructions.bprod l s = true <-> forall (i : W), i in l -> s i = true
```

Axiom dependencies: propext, Quot.sound.

L074 CircuitObstructions.pointIndicator_true_iff

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 31.

```
CircuitObstructions.pointIndicator_true_iff {n : Nat} (v x : Fin n -> Bool) :
  CircuitObstructions.pointIndicator v x = true <-> x = v
```

Axiom dependencies: propext, Quot.sound.

L075 CircuitObstructions.pointIndicator_degree

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 44.

```
CircuitObstructions.pointIndicator_degree {n : Nat} (v : Fin n -> Bool) :
  CircuitObstructions.DegreeLE n (CircuitObstructions.pointIndicator v)
```

Axiom dependencies: propext, Quot.sound.

L076 CircuitObstructions.completeSafeDegree_le_inputs

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 56.

```
CircuitObstructions.completeSafeDegree_le_inputs {n : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.CompleteSafeDegreeLE f n
```

Axiom dependencies: propext, Quot.sound.

L077 CircuitObstructions.exists_least_degree

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 65.

```
CircuitObstructions.exists_least_degree (P : Nat -> Prop) (n : Nat) (hn : P n) : exists r, P r /\ forall (k : Nat), P
↪  k -> r <= k
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

SafeDegree (continued)

L078 CircuitObstructions.beta_exists

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 83.

```
CircuitObstructions.beta_exists {n : Nat} (f : CircuitObstructions.BFun n) :
  exists r, CircuitObstructions.CompleteSafeDegreeLE f r /\ forall (k : Nat),
  ↪  CircuitObstructions.CompleteSafeDegreeLE f k -> r <= k
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L079 CircuitObstructions.beta_spec

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 90.

```
CircuitObstructions.beta_spec {n : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.CompleteSafeDegreeLE f (CircuitObstructions.beta f)
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L080 CircuitObstructions.beta_le_of_cover

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 93.

```
CircuitObstructions.beta_le_of_cover {n d : Nat} {f : CircuitObstructions.BFun n}
  (h : CircuitObstructions.CompleteSafeDegreeLE f d) : CircuitObstructions.beta f <= d
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L081 CircuitObstructions.beta_le_iff

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 97.

```
CircuitObstructions.beta_le_iff {n d : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.beta f <= d <-> CircuitObstructions.CompleteSafeDegreeLE f d
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L082 CircuitObstructions.beta_le_inputs

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 103.

```
CircuitObstructions.beta_le_inputs {n : Nat} (f : CircuitObstructions.BFun n) : CircuitObstructions.beta f <= n
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L083 CircuitObstructions.beta_zero

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 106.

```
CircuitObstructions.beta_zero (n : Nat) : (CircuitObstructions.beta fun x => false) = 0
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

SafeDegree (continued)

L084 CircuitObstructions.canonical_cut_completeSafeDegree

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 113.

```
CircuitObstructions.canonical_cut_completeSafeDegree (n a : Nat)
  (p q : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) (p ++ q) f) :
  CircuitObstructions.CompleteSafeDegreeLE f
    (max (CircuitObstructions.toffoliCount p) (CircuitObstructions.toffoliCount q) + 1)
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L085 CircuitObstructions.Gate.toffoliCost_le_one

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 124.

```
CircuitObstructions.Gate.toffoliCost_le_one {W : Type} (g : CircuitObstructions.Gate W) : g.toffoliCost <= 1
```

Axiom dependencies: propext, Quot.sound.

L086 CircuitObstructions.toffoliCount_nil

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 127.

```
CircuitObstructions.toffoliCount_nil {W : Type} : CircuitObstructions.toffoliCount [] = 0
```

Axiom dependencies: propext.

L087 CircuitObstructions.toffoliCount_cons

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 129.

```
CircuitObstructions.toffoliCount_cons {W : Type} (g : CircuitObstructions.Gate W) (c : CircuitObstructions.Circuit W)
↪  :
  CircuitObstructions.toffoliCount (g :: c) = g.toffoliCost + CircuitObstructions.toffoliCount c
```

Axiom dependencies: propext.

L088 CircuitObstructions.exists_cut_with_toffoliCount

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 133.

```
CircuitObstructions.exists_cut_with_toffoliCount {W : Type} (c : CircuitObstructions.Circuit W) (k : Nat)
  (hk : k <= CircuitObstructions.toffoliCount c) : exists p q, c = p ++ q /\ CircuitObstructions.toffoliCount p = k
```

Axiom dependencies: propext, Quot.sound.

SafeDegree (continued)

L089 CircuitObstructions.exists_balanced_toffoli_cut

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 155.

```
CircuitObstructions.exists_balanced_toffoli_cut {W : Type} (c : CircuitObstructions.Circuit W) :
  exists p q,
    c = p ++ q /\
      max (CircuitObstructions.toffoliCount p) (CircuitObstructions.toffoliCount q) <=
        (CircuitObstructions.toffoliCount c + 1) / 2
```

Axiom dependencies: propext, Quot.sound.

L090 CircuitObstructions.canonical_restored_completeSafeDegree

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 165.

```
CircuitObstructions.canonical_restored_completeSafeDegree (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  CircuitObstructions.CompleteSafeDegreeLE f ((CircuitObstructions.toffoliCount c + 1) / 2 + 1)
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L091 CircuitObstructions.beta_le_half_toffoliCount

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 173.

```
CircuitObstructions.beta_le_half_toffoliCount (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  CircuitObstructions.beta f <= (CircuitObstructions.toffoliCount c + 1) / 2 + 1
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L092 CircuitObstructions.beta_restoration_bound

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 179.

```
CircuitObstructions.beta_restoration_bound (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  2 * CircuitObstructions.beta f <= CircuitObstructions.toffoliCount c + 3
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L093 CircuitObstructions.toffoliCount_ge_two_beta_sub_three

Block 02. Source: CircuitObstructions/SafeDegree.lean, line 186.

```
CircuitObstructions.toffoliCount_ge_two_beta_sub_three (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  2 * CircuitObstructions.beta f - 3 <= CircuitObstructions.toffoliCount c
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

Declaration register: AffineOutput

14 DECLARATIONS

New zero-Toffoli affine circuit construction, affine-output optimization, and the joint main endpoint.

L094 CircuitObstructions.restores_append

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 12.

```
CircuitObstructions.restores_append {X W : Type} [DecidableEq W] (e : CircuitObstructions.Boundary X W)
  (c d : CircuitObstructions.Circuit W) (f g : X -> Bool) (hc : CircuitObstructions.Restores e c f)
  (hd : CircuitObstructions.Restores e d g) : CircuitObstructions.Restores e (c ++ d) fun x => f x ^^ g x
```

Axiom dependencies: propext.

L095 CircuitObstructions.canonical_target_not

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 20.

```
CircuitObstructions.canonical_target_not (n a : Nat) (x : Fin n -> Bool) (b : Bool) :
  (CircuitObstructions.Gate.not (CircuitObstructions.canonicalBoundary n a).target).eval
      ((CircuitObstructions.canonicalBoundary n a).encode x b) =
    (CircuitObstructions.canonicalBoundary n a).encode x (b ^^ true)
```

Axiom dependencies: propext, Quot.sound.

L096 CircuitObstructions.canonical_input_to_target

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 34.

```
CircuitObstructions.canonical_input_to_target (n a : Nat) (i : Fin n) (x : Fin n -> Bool) (b : Bool) :
  (CircuitObstructions.inputToTarget n a i).eval ((CircuitObstructions.canonicalBoundary n a).encode x b) =
    (CircuitObstructions.canonicalBoundary n a).encode x (b ^^ x i)
```

Axiom dependencies: propext, Quot.sound.

L097 CircuitObstructions.affine_restored_exists

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 45.

```
CircuitObstructions.affine_restored_exists (n a : Nat) (g : CircuitObstructions.BFun n)
  (hg : CircuitObstructions.DegreeLE 1 g) :
  exists d,
    CircuitObstructions.toffoliCount d = 0 /\
      CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) d g
```

Axiom dependencies: propext, Quot.sound.

L098 CircuitObstructions.betaStar_exists

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 76.

```
CircuitObstructions.betaStar_exists {n : Nat} (f : CircuitObstructions.BFun n) :
  exists r, CircuitObstructions.AffineSafeDegreeLE f r /\ forall (k : Nat), CircuitObstructions.AffineSafeDegreeLE f
  ↪  k -> r <= k
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

AffineOutput (continued)

L099 CircuitObstructions.betaStar_spec

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 85.

```
CircuitObstructions.betaStar_spec {n : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.AffineSafeDegreeLE f (CircuitObstructions.betaStar f)
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L100 CircuitObstructions.betaStar_le_of_bound

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 88.

```
CircuitObstructions.betaStar_le_of_bound {n d : Nat} {f : CircuitObstructions.BFun n}
  (h : CircuitObstructions.AffineSafeDegreeLE f d) : CircuitObstructions.betaStar f <= d
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L101 CircuitObstructions.betaStar_le_inputs

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 92.

```
CircuitObstructions.betaStar_le_inputs {n : Nat} (f : CircuitObstructions.BFun n) : CircuitObstructions.betaStar f <=
↪  n
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L102 CircuitObstructions.beta_le_betaStar

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 97.

```
CircuitObstructions.beta_le_betaStar {n : Nat} (f : CircuitObstructions.BFun n) :
  CircuitObstructions.beta f <= CircuitObstructions.betaStar f
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L103 CircuitObstructions.betaStar_le_half_toffoliCount

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 101.

```
CircuitObstructions.betaStar_le_half_toffoliCount (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  CircuitObstructions.betaStar f <= (CircuitObstructions.toffoliCount c + 1) / 2 + 1
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

AffineOutput (continued)

L104 CircuitObstructions.betaStar_restoration_bound

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 112.

```
CircuitObstructions.betaStar_restoration_bound (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  2 * CircuitObstructions.betaStar f <= CircuitObstructions.toffoliCount c + 3
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L105 CircuitObstructions.toffoliCount_ge_two_betaStar_sub_three

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 119.

```
CircuitObstructions.toffoliCount_ge_two_betaStar_sub_three (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  2 * CircuitObstructions.betaStar f - 3 <= CircuitObstructions.toffoliCount c
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L106 CircuitObstructions.canonical_restored_output_degree

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 126.

```
CircuitObstructions.canonical_restored_output_degree (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  CircuitObstructions.DegreeLE (CircuitObstructions.toffoliCount c + 1) f
```

Axiom dependencies: propext, Quot.sound.

L107 CircuitObstructions.restored_degree_and_cover_bound

Block 02. Source: CircuitObstructions/AffineOutput.lean, line 140.

```
CircuitObstructions.restored_degree_and_cover_bound (n a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire n a)) (f : CircuitObstructions.BFun n)
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary n a) c f) :
  CircuitObstructions.DegreeLE (CircuitObstructions.toffoliCount c + 1) f /\
    2 * CircuitObstructions.betaStar f <= CircuitObstructions.toffoliCount c + 3
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

Declaration register: DegreeChecks

8 DECLARATIONS

New exact AND2 example and proof controls. General large-family optimality is not inferred from this example.

L108 CircuitObstructions.squareParity_add

Block 02. Source: CircuitObstructions/DegreeChecks.lean, line 17.

```
CircuitObstructions.squareParity_add (f g : CircuitObstructions.BFun 2) :
  (CircuitObstructions.squareParity fun x => f x ^^ g x) =
    (CircuitObstructions.squareParity f ^^ CircuitObstructions.squareParity g)
```

Axiom dependencies: propext.

L109 CircuitObstructions.affine_squareParity_zero

Block 02. Source: CircuitObstructions/DegreeChecks.lean, line 22.

```
CircuitObstructions.affine_squareParity_zero (f : CircuitObstructions.BFun 2) (h : CircuitObstructions.DegreeLE 1 f)
↪  :
  CircuitObstructions.squareParity f = false
```

Axiom dependencies: propext, Quot.sound.

L110 CircuitObstructions.andTwo_degree_two

Block 02. Source: CircuitObstructions/DegreeChecks.lean, line 39.

```
CircuitObstructions.andTwo_degree_two : CircuitObstructions.DegreeLE 2 CircuitObstructions.andTwo
```

Axiom dependencies: propext, Quot.sound.

L111 CircuitObstructions.andTwo_not_affine

Block 02. Source: CircuitObstructions/DegreeChecks.lean, line 43.

```
CircuitObstructions.andTwo_not_affine : not CircuitObstructions.DegreeLE 1 CircuitObstructions.andTwo
```

Axiom dependencies: propext, Quot.sound.

DegreeChecks (continued)

L112 CircuitObstructions.andTwo_no_degree_one_safe_cover

Block 02. Source: CircuitObstructions/DegreeChecks.lean, line 49.

```
CircuitObstructions.andTwo_no_degree_one_safe_cover :
  not CircuitObstructions.CompleteSafeDegreeLE CircuitObstructions.andTwo 1
```

Axiom dependencies: propext, Quot.sound.

L113 CircuitObstructions.beta_andTwo

Block 02. Source: CircuitObstructions/DegreeChecks.lean, line 64.

```
CircuitObstructions.beta_andTwo : CircuitObstructions.beta CircuitObstructions.andTwo = 2
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L114 CircuitObstructions.betaStar_andTwo

Block 02. Source: CircuitObstructions/DegreeChecks.lean, line 72.

```
CircuitObstructions.betaStar_andTwo : CircuitObstructions.betaStar CircuitObstructions.andTwo = 2
```

Axiom dependencies: propext, Classical.choice, Quot.sound.

L115 CircuitObstructions.restored_andTwo_needs_toffoli

Block 02. Source: CircuitObstructions/DegreeChecks.lean, line 78.

```
CircuitObstructions.restored_andTwo_needs_toffoli (a : Nat)
  (c : CircuitObstructions.Circuit (CircuitObstructions.CanonicalWire 2 a))
  (hR : CircuitObstructions.Restores (CircuitObstructions.canonicalBoundary 2 a) c CircuitObstructions.andTwo) :
  1 <= CircuitObstructions.toffoliCount c
```

Axiom dependencies: propext, Classical.choice, Quot.sound.